



BASE DE DATOS AVANZADAS Y BIG DATA

UPN.EDU.PE

Semana 04



Disparadores

Transacciones: commit y rollback



Presentación de la sesión

Logro de sesión

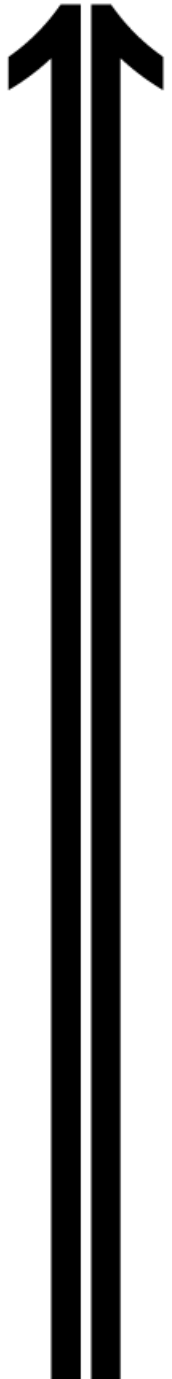
Al final de la sesión el alumno podrá crear disparadores con criterio y lógica , también comprenderá como se ejecutan las transacciones en SQL Server, para la aplicación de la lógica de los procesos de negocios.

Temario

- Disparadores (TRIGGERS)
- Transacciones (Commit, Rollback)



¿Qué es un trigger?



LANGUAGE DE CONTROL DE
TRANSACCIONES (TCL)

LANGUAGE DE CONTROL DE TRANSACCIONES (TCL)



¿Qué es una transacción?

- Una transacción es un conjunto de operaciones Transact SQL que se ejecutan como un único bloque. es decir, todas las declaraciones deben ejecutarse correctamente o ninguna de las declaraciones debe ejecutarse.
- La transacción más simple en SQL Server es una única sentencia SQL. una transacción 'autocommit', una transacción auto completada, se denomina transacción implícita.
- Las declaraciones de TCL disponibles en SQL SERVER son:

BEGIN TRANSACTION: Inicia una transacción explícita.

COMMIT TRANSACTION: Hacer que los cambios realizados en la transacción sean permanentes.

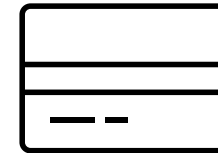
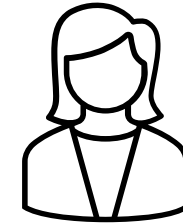
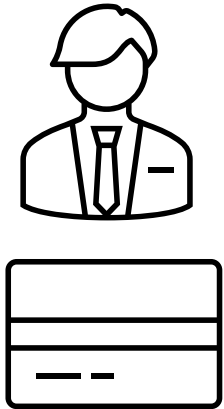
ROLLBACK TRANSACTION : Revierte el estado de la base de datos al último punto de confirmación.

SAVE TRANSACTION : Úselo para especificar un punto en la transacción al que luego puede revertir.

LANGUAGE DE CONTROL DE TRANSACCIONES (TCL)



EJEMPLO: Operación transferencia bancaria



```
CREATE TABLE CUENTAS
(
    numCuenta VARCHAR(12),
    saldo DECIMAL(18,2)
)
```

LENGUAGE DE CONTROL DE TRANSACCIONES (TCL)



```
--bloque para simular la operación
DECLARE @Monto DECIMAL(18,2), @CuentaADecrementar VARCHAR(12), @CuentaAIncrementar VARCHAR(12)

/* Asignamos el monto de la transacción y las cuentas a afectar*/
SET @Monto = 1900
SET @CuentaADecrementar = '20161206'
SET @CuentaAIncrementar = '20161207'

BEGIN TRANSACTION
BEGIN TRY
    /* Descontamos el monto de la cuenta a decrementar */
    UPDATE CUENTAS SET SALDO = SALDO - @Monto WHERE NUMCUENTA = @CuentaADecrementar
    /* Incrementamos el monto de la cuenta a incrementar */
    UPDATE CUENTAS SET SALDO = SALDO + @Monto WHERE NUMCUENTA = @CuentaAIncrementar
    /* Confirmamos la transaccion*/
    COMMIT TRANSACTION
END TRY
BEGIN CATCH
    /* Ocurrió un error, deshacemos los cambios*/
    ROLLBACK TRANSACTION
    PRINT 'Ha ocurrido un error!'
END CATCH
```

LENGUAGE DE CONTROL DE TRANSACCIONES (TCL)

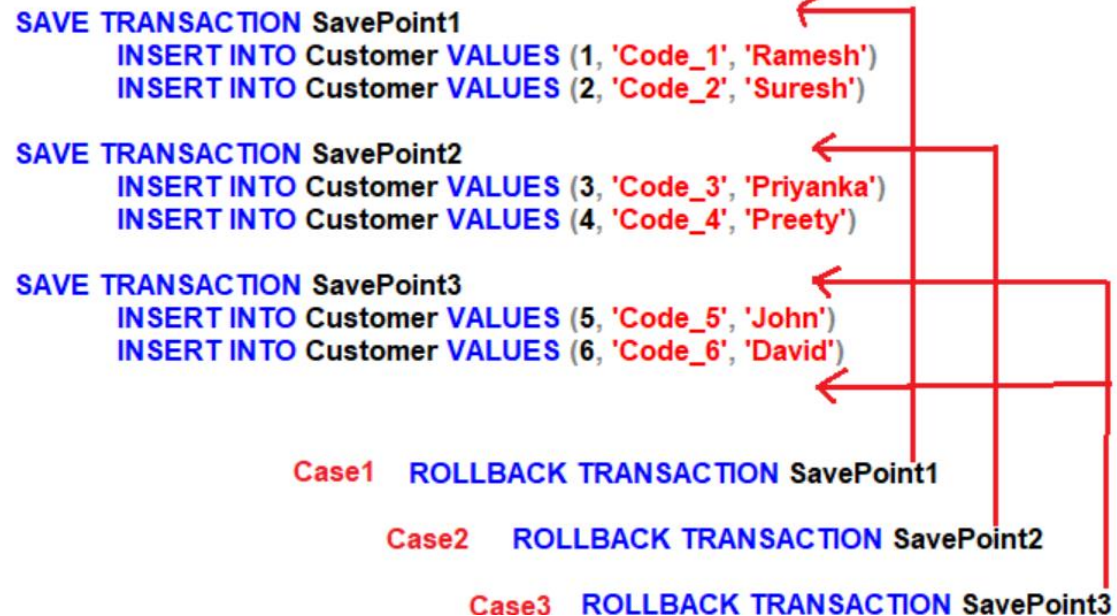


SAVEPOINT (SAVE TRANSACTION)

- Especifique un punto en una transacción al que luego pueda revertir.

EJEMPLO:

BEGIN TRANSACTION



LANGUAGE DE CONTROL DE TRANSACCIONES (TCL)



```
BEGIN TRAN
```

```
UPDATE Clientes SET Estado = 'Inactivo' WHERE iddepartamento = 1020
```

```
UPDATE Clientes SET Estado = 'DeBaja' WHERE iddepartamento=7025
```

```
SAVE TRANSACTION Punto1 -- Guardamos la transaccion (Savepoint)
```

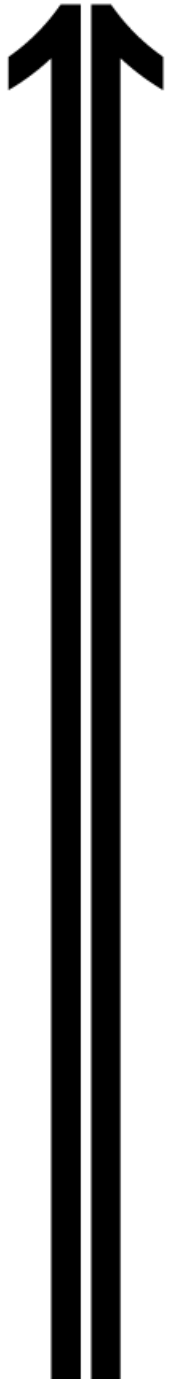
```
UPDATE Clientes SET Descripcion = 'Ninguna' WHERE idcliente=5896
```

```
-- Este ROLLBACK afecta solo a las instrucciones posteriores al savepoint Punto1.
```

```
ROLLBACK TRANSACTION Punto1
```

```
-- Confirmamos la transaccion
```

```
COMMIT
```



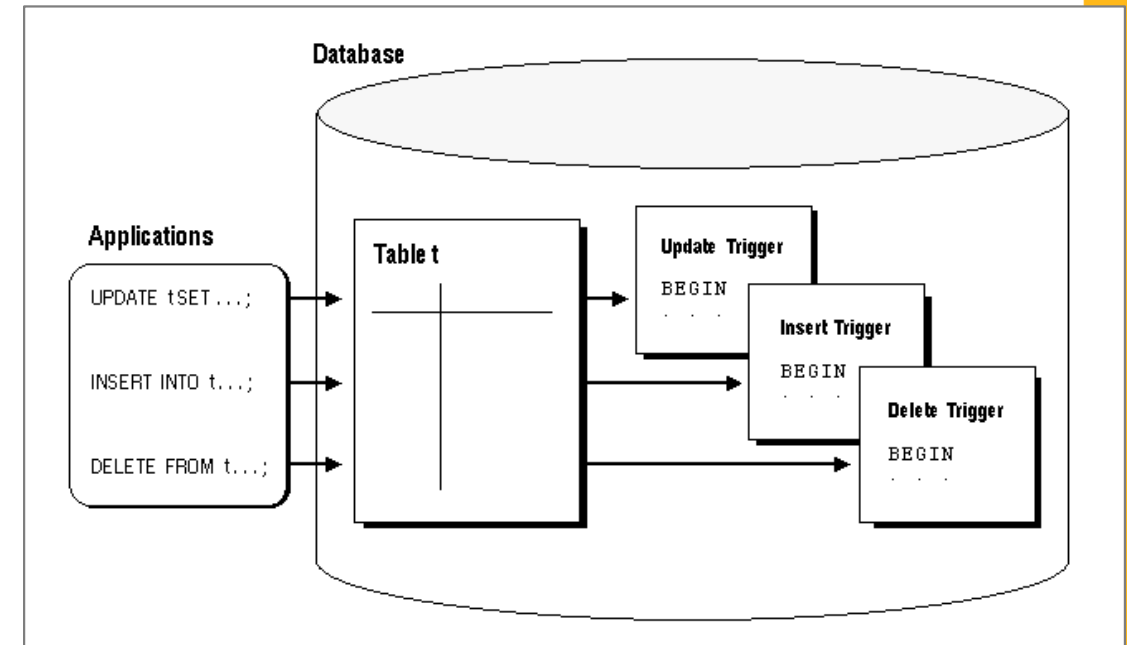
DISPARADORES (TRIGGERS)

DISPARADORES (TRIGGERS)



¿Qué es un TRIGGER?

- Un trigger (desencadenador o disparador) es una clase especial de procedimiento almacenado que se ejecuta automáticamente cuando se produce un evento en el servidor de bases de datos

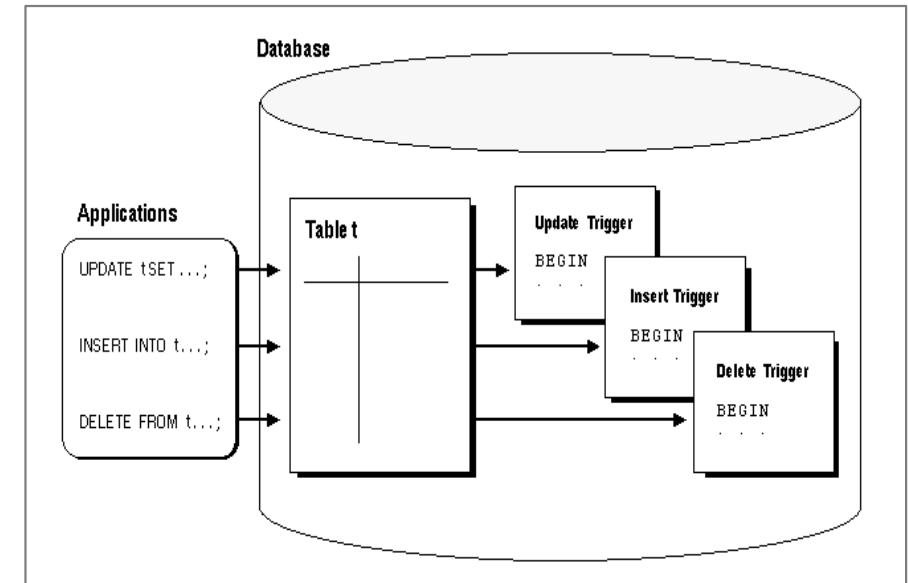


DISPARADORES (TRIGGERS)



SQL Server proporciona los siguientes tipos de triggers:

- Trigger **DML**, se ejecutan cuando se intenta modificar datos mediante un evento de lenguaje de manipulación de datos (DML). Los eventos DML son instrucciones INSERT, UPDATE o DELETE de una tabla o vista.
- Trigger **DDL**, se ejecutan en respuesta a una variedad de eventos de lenguaje de definición de datos (DDL).
- El código contenido en un disparador, denominado cuerpo del disparador, está formado por bloques TRANSACT SQL.

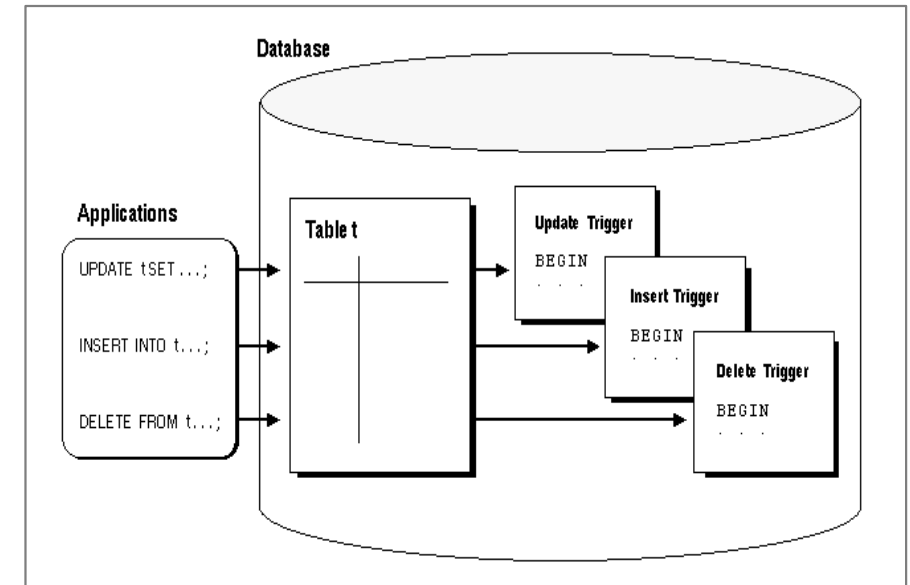


DISPARADORES (TRIGGERS)



SQL Server proporciona los siguientes tipos de triggers:

- Los eventos DML son instrucciones INSERT, UPDATE o DELETE de una tabla o vista. Estos desencadenadores se activan cuando se desencadena cualquier evento **válido**, con independencia de que las filas de la tabla se vean o no afectadas.
- Los desencadenadores DDL se ejecutan en respuesta a una variedad de eventos de lenguaje de definición de datos (DDL). Estos eventos corresponden principalmente a instrucciones CREATE, ALTER y DROP de Transact-SQL.



DISPARADORES (TRIGGERS)



Los triggers se utilizan comúnmente para garantizar la integridad de los datos (**Reglas complejas**), para **auditar** los cambios realizados en una tabla, para mantener **copias de seguridad** o para automatizar ciertas tareas administrativas. En general, los triggers son una herramienta muy útil para mejorar la eficiencia y la confiabilidad de las operaciones de base de datos en SQL Server.



DISPARADORES (TRIGGERS)



Creación / Modificación de disparador

```
CREATE [ OR ALTER ] TRIGGER [ schema_name . ]trigger_name
ON { table }
[ WITH <dml_trigger_option> [ ,...n ] ]
{ FOR | AFTER } { [ INSERT ] [ , ] [ UPDATE ] [ , ] [ DELETE ] }
AS { sql_statement [ ; ] [ ,...n ] }
```

```
CREATE OR ALTER TRIGGER trgEjemploDML
ON tb_ejemplo
FOR INSERT , UPDATE
AS
Begin PRINT 'Trigger DML' End
```

```
CREATE [ OR ALTER ] TRIGGER trigger_name
ON { ALL SERVER | DATABASE }
[ WITH <ddl_trigger_option> [ ,...n ] ]
{ FOR | AFTER } { event_type | event_group } [ ,...n ]
AS { sql_statement [ ; ] [ ,...n ] | EXTERNAL NAME <method
specifier > [ ; ] }
```

```
CREATE TRIGGER trgEjemploDDL
ON DATABASE
FOR DROP TABLE
AS
Begin PRINT 'Trigger DDL' End
```

DISPARADORES (TRIGGERS)



Eliminación del disparador

```
DROP TRIGGER nombre_disparador;
```

Activar/Desactivar del disparador

```
ENABLE TRIGGER nombre_disparador ON tabla
```

```
DISABLE TRIGGER nombre_disparador ON tabla
```

DISPARADORES (TRIGGER DML)



Implemente un trigger DML en la tabla Distritos, que muestre un mensaje cada vez que se realice una inserción o actualización o eliminación.

```
CREATE TRIGGER trgDistrito
ON RRHH.Distritos
FOR INSERT, UPDATE, DELETE
AS
BEGIN
    PRINT 'SE REALIZÓ UNA OPERACION DML'
END
```

```
1 CREATE OR ALTER TRIGGER trgDistrito
2 ON RRHH.Distritos
3 FOR INSERT, UPDATE, DELETE
4 AS
5 BEGIN
6     PRINT 'SE REALIZÓ UNA OPERACION DML'
7 END
```

Messages

23:53:50	Started executing query at Line 1 Commands completed successfully. Total execution time: 00:00:00.006
----------	---

DISPARADORES (TRIGGER DML)



Comprobamos el efecto del desencadenador o trigger cuando insertamos una nueva categoría.

```
SET NOCOUNT ON  
INSERT INTO RRHH.Distritos VALUES(6, 'Breña')
```

1 SET NOCOUNT ON

2 INSERT INTO RRHH.Distritos VALUES(6, 'Breña')

Messages

23:57:48

Started executing query at Line 1

SE REALIZÓ UNA OPERACION DML ←

Total execution time: 00:00:00.004

DISPARADORES (TRIGGERS)



Registros DELETED y INSERTED

- En las instrucciones de desencadenadores **DML** se usan dos tablas especiales: la tabla ***inserted*** y la tabla ***deleted***. SQL Server crea y administra automáticamente ambas tablas. Puede utilizar estas tablas temporales residentes en memoria para probar los efectos de determinadas modificaciones de datos y para establecer condiciones para las acciones de los desencadenadores DML.

EVENTO	PSEUDO REGISTROS	
	DELETED	INSERTED
INSERT	NULL	Nuevos valores
DELETE	Valores almacenados	NULL
UPDATE	Valores almacenados	Nuevos valores

DISPARADORES (TRIGGERS)



Ejemplo de disparadores

```
CREATE OR ALTER TRIGGER [RRHH].[trgDistrito]
ON [RRHH].[Distritos]
FOR INSERT, UPDATE, DELETE
AS
BEGIN
    SELECT * FROM inserted
    SELECT * FROM deleted
END
GO
```

DISPARADORES (TRIGGERS)



Ejemplo de disparadores

13
14

INSERT INTO RRHH.Distritos VALUES(99,'SJL')

ResultsMessages

	idDistrito	nomDistrito
1	99	SJL

	idDistrito	nomDistrito
--	------------	-------------

13
14
15
16

UPDATE RRHH.Distritos
SET nomDistrito = 'VMT'
WHERE idDistrito = 99

ResultsMessages

	idDistrito	nomDistrito
1	99	VMT

	idDistrito	nomDistrito
1	99	SJL

13
14
15

DELETE RRHH.Distritos
WHERE idDistrito = 99

ResultsMessages

	idDistrito	nomDistrito
--	------------	-------------

	idDistrito	nomDistrito
1	99	VMT

```
INSERT INTO RRHH.Distritos  
VALUES(99,'SJL')
```

```
UPDATE RRHH.Distritos  
SET nomDistrito = 'VMT'  
WHERE idDistrito = 99
```

```
DELETE RRHH.Distritos  
WHERE idDistrito = 99
```

DISPARADORES (TRIGGERS)



Ejemplo 1

- Crear un **TRIGGER** en el esquema **RRHH** con el nombre **TRG_INS_CARGO**, que se va a ejecutar cada vez que se realiza un registro en la tabla **CARGOS**.
- Dentro del **TRIGGER TRG_INS_CARGO**, se registra en el siguiente mensaje en la tabla **LOG_TABLE** 'INSERCIÓN EN LA TABLA CARGOS' y el usuario del Sistema Operativo que realizó la inserción (use **SUSER_NAME()**).
- La tabla **LOG_TABLE** , tiene siguientes columnas:
 - LOG_COLUMN VARCHAR2(200)
 - USER_NAME VARCHAR2(20)

DISPARADORES (TRIGGERS)



Ejemplo 1

```
CREATE TABLE LOG_TABLE
(
  LOG_COLUMN VARCHAR(200) ,
  USER_NAME VARCHAR(30)
)
```

```
CREATE OR ALTER TRIGGER RRHH.TRG_INS_CARGO
ON RRHH.CARGOS
FOR INSERT
AS
BEGIN
  INSERT INTO LOG_TABLE VALUES('INSERCIÓN EN LA TABLA CARGOS', SUSER_NAME())
END
```

RETO: Incluya el registro de las tres operaciones

DISPARADORES (TRIGGERS)



Ejemplo 2

Crear un TRIGGER en el esquema RRHH con nombre TRG_UPD_PRODUCTO, en la cual solo permita actualizar el **precioUnidad** de **PRODUCTOS** en horario de 8:00 a 12:00, caso contrario revertir la transacción y devolver un mensaje usando RAISERROR con severidad 10.

```
CREATE OR ALTER TRIGGER COMPRAS.TRG_UPD_PRODUCTO
ON Compras.productos
FOR UPDATE
AS
BEGIN
    IF DATEPART(HH,getdate()) NOT BETWEEN 8 AND 12
    BEGIN
        PRINT 'SON LAS' + STR(DATEPART(HH,getdate())) + ' HORAS'
        ROLLBACK TRANSACTION
        RAISERROR('!!SE ENCUENTRA FUERA DEL HORARIO PARA ACTUALIZAR',10,1)
    END
END
```

DISPARADORES (TRIGGERS)



Ejemplo 3

Modificar el **TRIGGER** con nombre **TRG_UPD_PRODUCTO**, en el cual se incluya la instrucción INSERT y cambie la severidad de RAISERROR para que genere una excepción.

```
CREATE OR ALTER TRIGGER COMPRAS.TRG_UPD_PRODUCTO
ON Compras.productos
FOR UPDATE, INSERT
AS
BEGIN
    IF DATEPART(HH,getdate()) NOT BETWEEN 8 AND 12
    BEGIN
        PRINT 'SON LAS' + STR(DATEPART(HH,getdate())) + ' HORAS'
        ROLLBACK TRANSACTION
        RAISERROR('!!SE ENCUENTRA FUERA DEL HORARIO PARA ACTUALIZAR',16,1)
    END
END
```

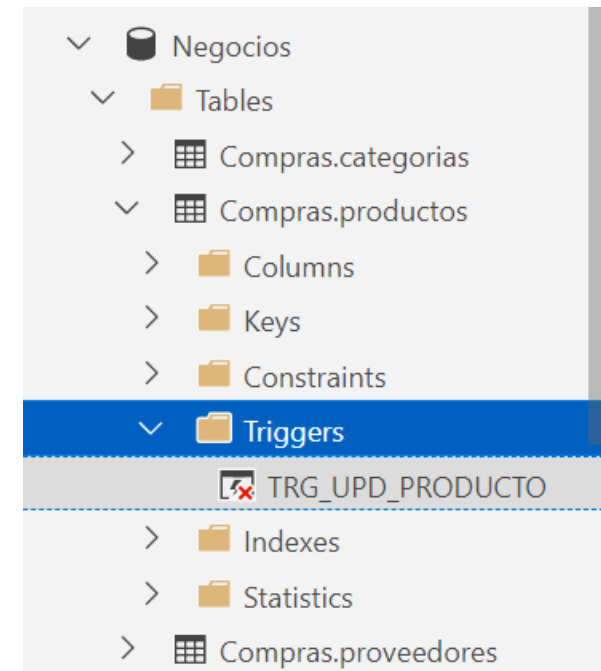
DISPARADORES (TRIGGERS)



Ejemplo 4

DEACTIVE el trigger para que permita actualizar el precio de unidad y verifique que se pueda actualizar.

```
DISABLE TRIGGER COMPRAS.TRG_UPD_PRODUCTO ON COMPRAS. PRODUCTOS
```



DISPARADORES (TRIGGERS)



Ejemplo 5

Por regla de negocio se ha definido que no se puede incrementar el precio de unidad de los productos más del 30% del precio actual. Cree un trigger que permita asegurar esta regla, caso contrario revertir la transacción y devolver un mensaje usando RAISERROR, con severidad 10, indicando cual es el valor máximo que se puede actualizar

DISPARADORES (TRIGGERS)



```
CREATE OR ALTER TRIGGER COMPRAS.TRG_UPD30_PRODUCTO
ON Compras.productos
FOR UPDATE
AS
BEGIN
    DECLARE @v_precioActual decimal(10,0), @v_precioNuevo decimal(10,0), @v_mensaje VARCHAR(200)

    SELECT @v_precioActual = PrecioUnidad FROM deleted
    SELECT @v_precioNuevo = PrecioUnidad FROM inserted
    IF @v_precioNuevo > 1.3 * @v_precioActual
    BEGIN
        set @v_mensaje = '!!NO SE PUEDE ACTUALIZAR MÁS DEL 30 porciento AL PRECIO DE UNIDAD. MÁXIMO : ' +
TRIM(STR(1.3 * @v_precioActual))
        ROLLBACK TRANSACTION
        RAISERROR(@v_mensaje,16,1)
    END
END

END
```

DISPARADORES (TRIGGERS)



Ejemplo 6

Implemente un trigger **DDL** que impida hacer cambios o eliminación a las tablas de la Base de Datos Negocios, por un tema de seguridad.

```
CREATE OR ALTER TRIGGER trgImpideEliminarTablas
ON DATABASE
AFTER DROP_TABLE
AS
BEGIN
    RAISERROR('!!NO PUEDE ELIMINAR UNA TABLA EN ESTA BD',16,1)
    ROLLBACK
END
```



¿Preguntas o comentarios?



CONCLUSIONES

Un desencadenante en SQLSERVER se ejecuta automáticamente cuando se produce un evento desencadenante.

Los triggers pueden definirse para las operaciones INSERT, DELETE o UPDATE.

Una misma tabla puede tener varios triggers asociados. En tal caso es necesario conocer el orden en el que se van a ejecutar.

Los disparadores se activan al ejecutarse la sentencia SQL.

Podemos tratar un conjunto de operaciones como una unidad por medio del manejo de transacciones, haciendo uso del TCL

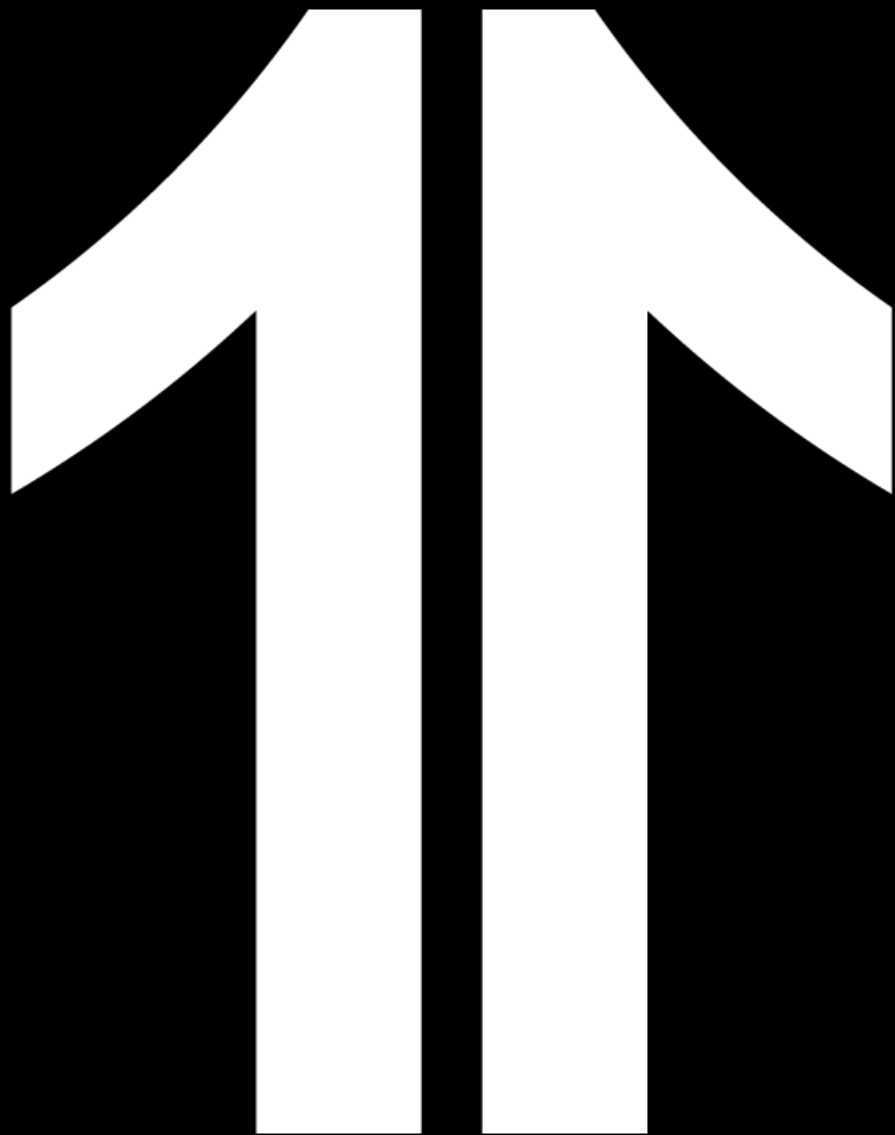


REFERENCIAS BIBLIOGRÁFICAS



SQLSERVER Guide and Reference

- CREATE TRIGGER (<https://learn.microsoft.com/es-es/sql/t-sql/statements/create-trigger-transact-sql?view=sql-server-ver16>)
- TRIGGER DML(<https://learn.microsoft.com/es-es/sql/relational-databases/triggers/dml-triggers?view=sql-server-ver16>)
- TRIGGER DDL(<https://learn.microsoft.com/es-es/sql/relational-databases/triggers/ddl-triggers?view=sql-server-ver16>)
- TRASACCIONES (<https://learn.microsoft.com/es-es/sql/t-sql/language-elements/transactions-transact-sql?view=sql-server-ver16>)



GRACIAS

